Screening simulations - continuous nonlinear outcome (mfscreen)

J. Dedecker, M.-L. Taupin and A.-S. Tocquet

2026-06-29

Contents

1 Objective

This document evaluates the model-free marginal screening procedure on $p = 2000$ explanatory variables. Variables $X^{(1)}, \ldots, X^{(10)}$ generate the response; $X^{(11)}, \ldots, X^{(14)}$ are noisy versions of signal variables and are therefore correlated with the response; and $X^{(15)}, \ldots, X^{(18)}$, together with $X^{(19)}, \ldots, X^{(2000)}$, are null variables for screening.

The first five variables are centered Gaussian variables with Toeplitz covariance matrix $\Sigma_{jk} = \rho^{|j-k|}$, where $\rho = 0.57$. The additional noise variables use $Z_3 \sim \mathcal{N}(0, 0.4)$, $Z_4 \sim \mathcal{N}(0, 0.02)$, $Z_5 \sim \mathcal{N}(0, 0.35)$, and $Z_6 \sim \mathcal{N}(0, 0.55)$. The second arguments are the standard deviations supplied to `rnorm()`.

2 Screening with `mfscreen`

This version uses the Rcpp routine `mfscreen::screening_test_matrix()` rather than explicitly computing marginal slopes, residuals, and White variance estimators in R.

For each explanatory variable $X^{(j)}$, the routine returns the distance score

$$D_j^2 = \frac{\widehat{V}^{(j)}}{n\{\widehat{\tau}^{(j)}\}^2} = \frac{1}{\{\widehat{\gamma}^{(j)}\}^2}.$$

For a target false-positive rate `q`, the variable is selected when

$$D_j^2 \leq \frac{1}{\{\Phi^{-1}(1 - q/2)\}^2}.$$

`screening_test_matrix()` returns `scores`, `threshold`, `gamma`, `selected`, and `selected_indices`. All Monte Carlo summaries below are computed directly from `selected`; the former rule based on `abs(gamma_chapo)` must not be applied to the new scores.

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE, warning = FALSE, message = FALSE)

if (!requireNamespace("mfscreen", quietly = TRUE)) {
  stop(
    "The mfscreen package must be installed before running this document. ",
    "For example: install.packages('mfscreen_0.1.1.tar.gz', repos = NULL, type = 'source')."
  )
}
```

3 Design parameters and common functions

```r
# Common design parameters.
p_total <- 2000L
p_signal <- 10L
p_proxy_or_null <- 8L
p_gaussian_noise <- p_total - p_signal - p_proxy_or_null
stopifnot(p_gaussian_noise == 1982L)

theta_star <- c(
  -1,
  3.619350, -3.274923, 2.963273, -2.681280,
  2, 4, 6, 3, 2, 4
)
stopifnot(length(theta_star) == p_signal + 1L)

rho <- 0.57
rho_noise <- 0.70
q <- 0.10
N <- 500L

# Four separate Monte Carlo experiments will be run, one for each sample size.
sample_sizes <- c(500L, 1000L, 2000L, 2500L)
stopifnot(length(sample_sizes) == 4L)

variable_labels <- c(
  "X1", "X2", "X3", "X4", "X5",
  "X6_bern", "X7_chisq2", "X8=X1*P(2)",
  "X9=X2*N(1,1)", "X10=Student(14)",
  "X11=X1+Z3", "X12=X3+Z4", "X13=X4+Z5", "X14=X5+Z6",
  "X15=Z3", "X16=Z4", "X17=Z5", "X18=Z6",
  paste0("X", 19:p_total)
)
stopifnot(length(variable_labels) == p_total)

# Variables 1--14 are correlated with Y. Variables 15--2000 are null for
# the false-positive-rate calculation.
true_positive_idx <- 1:14
false_positive_idx <- 15:p_total

# Generate N(0, Sigma) data without external packages. If Z has rows N(0, I),
# Z %*% chol(Sigma) has covariance matrix Sigma.
simulate_gaussian_design <- function(n, p, rho, sd = 1) {
  Sigma <- sd^2 * toeplitz(rho^(0:(p - 1L)))
  Z <- matrix(rnorm(n * p), nrow = n, ncol = p)
  Z %*% chol(Sigma)
}

build_design <- function(n, p_gaussian_noise, rho, rho_noise) {
  X_first_five <- simulate_gaussian_design(n, p = 5L, rho = rho)
  colnames(X_first_five) <- paste0("X", 1:5)

  Z3 <- rnorm(n, mean = 0, sd = 0.40)
  Z4 <- rnorm(n, mean = 0, sd = 0.02)
```

```r
  Z5 <- rnorm(n, mean = 0, sd = 0.35)
  Z6 <- rnorm(n, mean = 0, sd = 0.55)

  X_signal <- cbind(
    X_first_five,
    X6_bern = rbinom(n, size = 1L, prob = 0.35),
    X7_chisq2 = rchisq(n, df = 2),
    `X8=X1*P(2)` = X_first_five[, 1L] * rpois(n, lambda = 2),
    `X9=X2*N(1,1)` = X_first_five[, 2L] * rnorm(n, mean = 1, sd = 1),
    `X10=Student(14)` = rt(n, df = 14)
  )

  X_noise <- simulate_gaussian_design(n, p = p_gaussian_noise, rho = rho_noise)

  X <- cbind(
    X_signal,
    `X11=X1+Z3` = X_first_five[, 1L] + Z3,
    `X12=X3+Z4` = X_first_five[, 3L] + Z4,
    `X13=X4+Z5` = X_first_five[, 4L] + Z5,
    `X14=X5+Z6` = X_first_five[, 5L] + Z6,
    `X15=Z3` = Z3,
    `X16=Z4` = Z4,
    `X17=Z5` = Z5,
    `X18=Z6` = Z6,
    X_noise
  )

  storage.mode(X) <- "double"
  colnames(X) <- variable_labels
  stopifnot(nrow(X) == n, ncol(X) == p_total)

  list(X = X, X_signal = X_signal)
}

run_monte_carlo <- function(N, fun, n_cores) {
  iterations <- seq_len(N)

  # mclapply is not available on Windows. In that case, use the same code
  # sequentially so that the document remains portable.
  if (.Platform$OS.type == "windows" || n_cores <= 1L) {
    return(lapply(iterations, fun))
  }

  parallel::mclapply(
    iterations,
    fun,
    mc.cores = n_cores,
    mc.set.seed = TRUE
  )
}
```

4 Continuous nonlinear heteroscedastic outcome

The continuous response is generated from

$$Y = |X\theta^*|^{0.8} + X^{(5)}\varepsilon/3, \qquad \varepsilon \sim \chi^2(3) - 3,$$

where ε is independent of the explanatory variables. This construction is nonlinear and has heteroscedastic noise.

```
simulate_one_dataset <- function(n_current) {
  design <- build_design(
    n = n_current,
    p_gaussian_noise = p_gaussian_noise,
    rho = rho,
    rho_noise = rho_noise
  )

  linear_predictor <- as.vector(
    theta_star[1L] + design$X_signal %*% theta_star[-1L]
  )
  epsilon <- rchisq(n_current, df = 3) - 3

  y <- abs(linear_predictor)^0.8 +
    design$X_signal[, "X5"] * epsilon / 3

  screening <- mfscreen::screening_test_matrix(
    X = design$X,
    y = as.numeric(y),
    q = q
  )

  stopifnot(length(screening$selected) == p_total)

  list(
    selected = as.logical(screening$selected),
    threshold = as.numeric(screening$threshold),
    gamma = as.numeric(screening$gamma),
    selected_indices = as.integer(screening$selected_indices)
  )
}
```

5 Four separate Monte Carlo experiments

The following code runs **four independent Monte Carlo experiments**: $n = 500$, $n = 1000$, $n = 2000$, and $n = 2500$. Each experiment uses 500 replicates. Their results are then assembled into the four numeric columns of the final Table 1-style output.

```
RNGkind("L'Ecuyer-CMRG")
seed_base <- 20260629L

# A design matrix for n = 2500 has 5 million entries. Cap the number of
# simultaneous workers to avoid excessive memory use.
detected_cores <- parallel::detectCores(logical = FALSE)
if (is.na(detected_cores)) detected_cores <- 1L
```

```r
n_cores <- max(1L, min(4L, detected_cores))

summarise_experiment <- function(simulations, n_current) {
  selection_matrix <- do.call(rbind, lapply(simulations, `[[`, "selected"))
  storage.mode(selection_matrix) <- "logical"
  colnames(selection_matrix) <- variable_labels

  thresholds <- vapply(simulations, `[[`, numeric(1), "threshold")
  gammas <- vapply(simulations, `[[`, numeric(1), "gamma")

  stopifnot(
    nrow(selection_matrix) == N,
    ncol(selection_matrix) == p_total,
    !anyNA(selection_matrix),
    isTRUE(all.equal(thresholds, rep(thresholds[1L], N))),
    isTRUE(all.equal(gammas, rep(gammas[1L], N)))
  )

  selection_rates <- colMeans(selection_matrix)

  list(
    n = n_current,
    selection_rates = selection_rates,
    true_positive_rate = mean(selection_matrix[, true_positive_idx, drop = FALSE]),
    false_positive_rate = mean(selection_matrix[, false_positive_idx, drop = FALSE]),
    mean_selected_set_size = mean(rowSums(selection_matrix)),
    gamma = gammas[1L],
    score_threshold = thresholds[1L]
  )
}

# This loop makes four distinct Monte Carlo calls: one for each value of n.
results_by_n <- vector("list", length(sample_sizes))
names(results_by_n) <- as.character(sample_sizes)

for (k in seq_along(sample_sizes)) {
  n_current <- sample_sizes[k]

  simulations_n <- run_monte_carlo(
    N = N,
    fun = function(iteration) {
      # The seed is unique to the pair (sample size, replicate), making all
      # four experiments reproducible and independent of each other.
      set.seed(seed_base + 10000L * n_current + iteration)
      simulate_one_dataset(n_current)
    },
    n_cores = n_cores
  )

  results_by_n[[k]] <- summarise_experiment(
    simulations = simulations_n,
    n_current = n_current
  )
```

```
}
```

6 Table 1-style summary

The final table has exactly four simulation columns: $n = 500$, $n = 1000$, $n = 2000$, and $n = 2500$. It reports the selection frequency of $X^{(1)}$ through $X^{(18)}$, the true-positive rate, false-positive rate, and mean selected-set size. This matches the structure of Table 1 in the accompanying manuscript.

```
table_row_labels <- c(
  sprintf("$X^{(%d)}$", 1:18),
  "True Positive Rate",
  "False Positive Rate",
  "mean selected-set size"
)

table_column_labels <- paste0("$n = ", sample_sizes, "$")

# The output of vapply is a 21 x 4 matrix: one explicit column for each n.
table_values <- vapply(
  results_by_n,
  FUN = function(summary_n) {
    c(
      summary_n$selection_rates[1:18],
      summary_n$true_positive_rate,
      summary_n$false_positive_rate,
      summary_n$mean_selected_set_size
    )
  },
  FUN.VALUE = numeric(length(table_row_labels))
)

stopifnot(
  identical(dim(table_values), c(length(table_row_labels), 4L)),
  all(is.finite(table_values))
)

# Build a data frame with one row-label column plus the four visible n columns.
table1_mfscreen <- data.frame(
  "Explanatory Variable" = table_row_labels,
  check.names = FALSE,
  stringsAsFactors = FALSE
)

for (k in seq_along(sample_sizes)) {
  table1_mfscreen[[table_column_labels[k]]] <- table_values[, k]
}

stopifnot(
  ncol(table1_mfscreen) == 5L,
  identical(
    colnames(table1_mfscreen),
    c("Explanatory Variable", table_column_labels)
  )
```

```
)

expected_mean_selected <- length(true_positive_idx) +
  q * length(false_positive_idx)

save(
  results_by_n,
  table_values,
  table1_mfscreen,
  file = "continuous_nonlinear_mfscreen_N500_q010_n500_n1000_n2000_n2500.RData"
)
```

For every sample size, the `mfscreen` score threshold is 0.369612 and $\gamma = 1.64485$. The expected mean selected-set size is $14 + 0.10 \times 1986 = 212.6$.

```
table_caption <- sprintf(
  "N = %d, q = %.2f, continuous nonlinear output",
  N,
  q
)

# Print one table with the explanatory-variable column plus four visible
# simulation columns: n = 500, n = 1000, n = 2000, and n = 2500.
table_format <- if (knitr::is_latex_output()) "latex" else "html"

knitr::kable(
  table1_mfscreen,
  format = table_format,
  digits = 3,
  align = c("l", "r", "r", "r", "r"),
  booktabs = knitr::is_latex_output(),
  escape = FALSE,
  caption = table_caption
)
```

Table 1: N = 500, q = 0.10, continuous nonlinear output

Explanatory Variable	$n = 500$	$n = 1000$	$n = 2000$	$n = 2500$
$X^{(1)}$	1.000	1.000	1.000	1.000
$X^{(2)}$	0.996	1.000	1.000	1.000
$X^{(3)}$	0.986	0.998	1.000	1.000
$X^{(4)}$	0.380	0.558	0.814	0.896
$X^{(5)}$	0.644	0.880	0.992	0.994
$X^{(6)}$	0.560	0.838	0.990	0.988
$X^{(7)}$	1.000	1.000	1.000	1.000
$X^{(8)}$	0.998	1.000	1.000	1.000
$X^{(9)}$	0.990	1.000	1.000	1.000
$X^{(10)}$	0.984	1.000	1.000	1.000
$X^{(11)}$	1.000	1.000	1.000	1.000
$X^{(12)}$	0.986	0.998	1.000	1.000
$X^{(13)}$	0.332	0.510	0.766	0.842
$X^{(14)}$	0.574	0.772	0.980	0.980
$X^{(15)}$	0.108	0.086	0.092	0.120
$X^{(16)}$	0.094	0.106	0.128	0.096
$X^{(17)}$	0.124	0.088	0.102	0.094
$X^{(18)}$	0.076	0.084	0.096	0.102
True Positive Rate	0.816	0.897	0.967	0.979
False Positive Rate	0.102	0.101	0.101	0.101
mean selected-set size	214.164	213.918	213.888	213.910